

Supercomputing Architecture & SLURM Job Scheduling Guide

Location: `day3/SLURM_AND_SUPERCOMPUTER_GUIDE.md`

Topic: High-Performance Computing (HPC) Clusters, Supercomputer Architecture, and SLURM Workload Manager

Table of Contents

1. High-Performance Computing (HPC) Overview
2. Architecture of a Supercomputer
 - Node Types: Login vs Compute Nodes
 - High-Speed Interconnects (InfiniBand)
 - Shared Storage (Lustre / GPFS)
 - System Topology Diagram
3. What is SLURM?
 - Full Form and Core Role
 - Key SLURM Daemons
4. Dissecting `gpu.sh` Line-by-Line
 - SLURM Directives (`#SBATCH`)
 - Diagnostic Commands
 - Job Execution
5. Bare Minimum SLURM Templates
 - Minimal GPU Job Script
 - Minimal Multi-Core CPU / MPI Job Script
6. Essential SLURM Commands Cheat Sheet
7. Supercomputer Best Practices & Golden Rules

1. High-Performance Computing (HPC) Overview

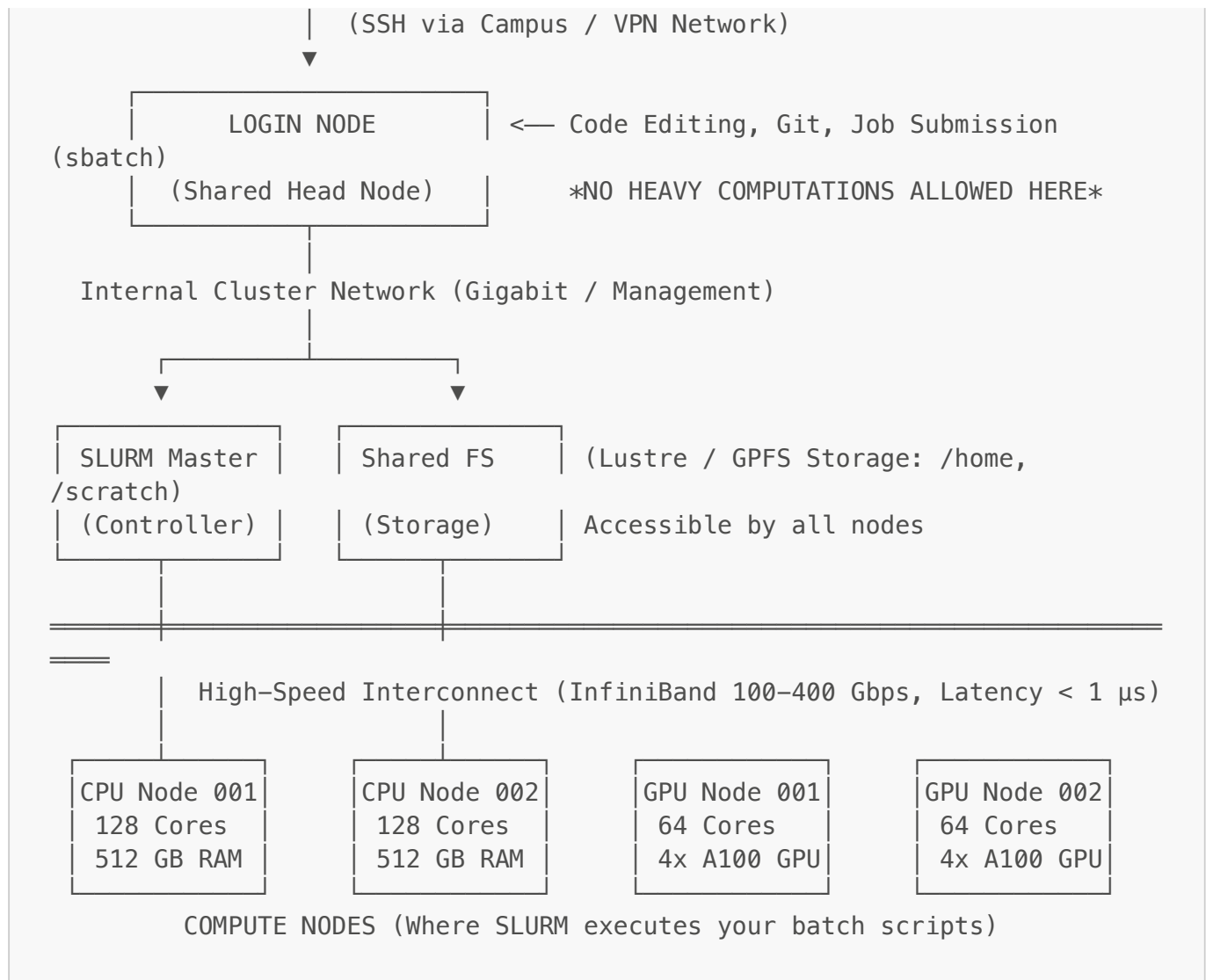
A **supercomputer** (or **HPC Cluster**) is not simply one gigantic, monolithic computer. Rather, it is an aggregation of hundreds or thousands of high-end standalone servers—called **Nodes**—tightly coupled together by an ultra-low-latency network fabric.

Instead of running an application on a single laptop with 8 or 16 CPU cores, an HPC cluster allows you to distribute computations across **tens of thousands of CPU cores and thousands of GPUs** simultaneously.

2. Architecture of a Supercomputer

SUPERCOMPUTER CLUSTER TOPOLOGY

[User Laptop / Workstation]



Node Types: Login vs Compute Nodes

1. Login Nodes (Head Nodes):

- The server you connect to when you type `ssh user@supercomputer.cdac.in`.
- Shared simultaneously by hundreds of users.
- **Purpose:** Compiling code, managing files, writing scripts, and submitting jobs.
- **Rule: Never execute heavy Python, MPI, or GPU scripts on the login node.** It will freeze the node for all other users and admins will terminate your processes.

2. Compute Nodes:

- Dedicated servers that perform the heavy calculations.
- Grouped into pools called **Partitions** (e.g., `cpu`, `gpu`, `highmem`, `debug`).
- Users cannot SSH into compute nodes directly without an active allocation granted by the scheduler.

High-Speed Interconnects (InfiniBand)

Standard office Ethernet has high latency (~20–50 microseconds) and bandwidth of 1 Gbps.

Supercomputers use specialized networking technology called **InfiniBand** (HDR / NDR):

- **Bandwidth:** 100 to 400 Gbps per link.
 - **Latency:** Sub-microsecond ($< 1 \mu\text{s}$).
 - **RDMA (Remote Direct Memory Access):** Process A on Node 1 can write directly into the RAM of Node 2 without interrupting Node 2's operating system kernel. This makes **MPI** (`mpi4py`) blazing fast.
-

Shared Storage (Lustre / GPFS)

Compute nodes typically do not store job data on their local disks. Instead, all nodes connect to a massive **Parallel File System** (e.g., **Lustre** or **IBM Spectrum Scale / GPFS**):

- When you edit a script in your `/home/username/` directory, that exact same file is instantly visible across every compute node.
 - Provides petabytes of storage with hundreds of gigabytes/second aggregate read/write throughput.
-

3. What is SLURM?

Full Form and Core Role

SLURM stands for:
Simple **L**inux **U**tility for **R**esource **M**anagement

SLURM is the world's most widely used **Workload Manager and Job Scheduler** for high-performance computing clusters and supercomputers (powering a majority of the TOP500 supercomputers).

SLURM performs three core functions:

1. **Resource Allocation:** Grants exclusive or shared access to compute resources (nodes, CPU cores, memory, GPUs) to users for a specific duration.
 2. **Framework for Execution:** Provides primitives to start, execute, and monitor jobs on allocated nodes (such as `srun` for MPI tasks).
 3. **Queue Arbitration:** Manages waiting queues of jobs, enforcing priorities, fair-share policies, and partition limits.
-

Key SLURM Daemons

- **`slurmctld` (Central Controller):** Runs on the management node. Monitors node states, tracks resource availability, and schedules queued jobs.
 - **`slurmd` (Node Daemon):** Runs on each compute node. Waits for instructions from `slurmctld`, allocates CPU/GPU cores, starts tasks, and reports health.
 - **`slurmdbd` (Database Daemon):** Records accounting logs, user quotas, bank hours, and historical job execution times.
-

4. Dissecting `gpu.sh` Line-by-Line

Here is the script you received:

```
#!/bin/bash
#SBATCH --job-name=gpu_check
#SBATCH --partition=gpu
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --gres=gpu:1
#SBATCH --time=00:05:00
#SBATCH --output=gpu_check_%j.out
#SBATCH --error=gpu_check_%j.err

echo "=====
echo "GPU CHECK"
echo "=====

echo "Hostname:"
hostname

echo
echo "Date:"
date

echo
echo "CUDA_VISIBLE_DEVICES:"
echo $CUDA_VISIBLE_DEVICES

echo
echo "NVIDIA-SMI:"
nvidia-smi

echo
echo "GPU Information:"
nvidia-smi --query-
gpu=index,name,driver_version,memory.total,memory.free,memory.used,utilizat
ion.gpu \
--format=csv

echo
echo "=====

python3 vinod/ex1.py
```

SLURM Directives (#SBATCH)

Even though lines begin with # (standard bash comments), the SLURM command `sbatch` reads `#SBATCH` as configuration parameters:

Directive	Meaning	Explanation
<code>#!/bin/bash</code>	Shebang	Specifies that this script should be executed using the Bash shell.

Directive	Meaning	Explanation
<code>#SBATCH --job-name=gpu_check</code>	Job Name	An arbitrary identifier that will show up when you check the queue using <code>squeue</code> .
<code>#SBATCH --partition=gpu</code>	Target Queue	Specifies the cluster partition. Supercomputers divide nodes into partitions (e.g. <code>gpu</code> , <code>standard</code> , <code>compute</code>). This routes your job to machines with physical GPUs installed.
<code>#SBATCH --nodes=1</code>	Node Count	Requests exactly 1 physical server.
<code>#SBATCH --ntasks=1</code>	Task Count	Launches 1 process. (For MPI jobs with 4 ranks, you would set <code>--ntasks=4</code>).
<code>#SBATCH --gres=gpu:1</code>	Generic Resources	Requests 1 physical GPU (<code>gpu:1</code>). On nodes with 4 or 8 GPUs, SLURM isolates and assigns exactly 1 GPU exclusively to your job.
<code>#SBATCH --time=00:05:00</code>	Wallclock Limit	<code>HH:MM:SS</code> . Requests a maximum run time of 5 minutes . If the job runs longer, SLURM will kill it. Shorter time limits help jobs start faster in the queue!
<code>#SBATCH --output=gpu_check_%j.out</code>	Standard Output	Captures all normal console prints (<code>print()</code> , <code>echo</code>). <code>%j</code> is replaced by the unique Job ID (e.g., <code>gpu_check_84210.out</code>).
<code>#SBATCH --error=gpu_check_%j.err</code>	Standard Error	Captures all error messages, Python tracebacks, and warnings (e.g., <code>gpu_check_84210.err</code>).

Diagnostic Commands

The middle portion of the script inspects the compute node before running Python:

- **hostname**: Prints the internal host name of the compute node SLURM assigned to you (e.g., `gpu04.cluster.local`).
- **date**: Prints the timestamp when execution began.
- **echo \$CUDA_VISIBLE_DEVICES**: SLURM assigns GPUs by setting this environment variable. If the physical node has GPUs 0, 1, 2, 3, and SLURM allocated GPU 2 to you, it sets `CUDA_VISIBLE_DEVICES=2`. Inside Python, CUDA will only see this assigned GPU as device `0`.
- **nvidia-smi**: NVIDIA System Management Interface utility. Prints a full hardware table showing driver version, CUDA version, temperature, power draw, and VRAM memory.
- **nvidia-smi --query-gpu=... --format=csv**: Extracts clean CSV columns for quick parsing of GPU model name, driver, total memory, and utilization percentage.

Job Execution

```
python3 vinod/ex1.py
```

This is the actual application command. Once the diagnostics print, Python is invoked on the compute node, executes `vinod/ex1.py` with access to the allocated GPU, and writes any output into `gpu_check_%j.out`.

5. Bare Minimum SLURM Templates

Minimal GPU Job Script

Save as `run_gpu.sh`:

```
#!/bin/bash
#SBATCH --job-name=my_gpu_job
#SBATCH --partition=gpu
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --gres=gpu:1
#SBATCH --time=00:15:00
#SBATCH --output=job_%j.out
#SBATCH --error=job_%j.err

# Optional: Load cluster environment modules if needed:
# module load cuda/12.2 python/3.11

# Run your Python program
python3 vinod/ex1.py
```

Minimal Multi-Core CPU / MPI Job Script

For Day 3 MPI programs across multiple cores/nodes (`run_mpi.sh`):

```
#!/bin/bash
#SBATCH --job-name=mpi_job
#SBATCH --partition=compute
#SBATCH --nodes=2
#SBATCH --ntasks=8
#SBATCH --time=00:10:00
#SBATCH --output=mpi_%j.out
#SBATCH --error=mpi_%j.err

# Optional: Load OpenMPI module
# module load openmpi
```

```
# Run with mpirun or srun across the 8 allocated ranks:
srun python3 day3/demo/01_mpi_point_to_point.py
```

6. Essential SLURM Commands Cheat Sheet

Command	Purpose	Example
sbatch <script>	Submit a batch script to the queue	sbatch gpu.sh
squeue -u \$USER	View all active & waiting jobs for your username	squeue -u vinod
scancel <job_id>	Cancel / terminate a running or queued job	scancel 84210
sinfo	View cluster partitions and node states (idle, alloc, down)	sinfo
srun	Run interactive tasks directly on compute nodes	srun --partition=gpu --gres=gpu:1 --pty bash
sacct -j <job_id>	View job accounting history (CPU time, exit code, memory used)	sacct -j 84210

Common SLURM Job State Codes (**squeue**)

- **PD (Pending)**: Job is waiting in the queue for requested nodes/GPUs to become available.
- **R (Running)**: Job is currently executing on compute nodes.
- **CG (Completing)**: Job is finishing up and transferring files back.
- **CD (Completed)**: Job finished successfully (exit code 0).
- **F (Failed)**: Job terminated with a non-zero exit code (check the **.err** file).
- **T0 (Timeout)**: Job exceeded its requested **--time** limit and was killed by SLURM.

7. Supercomputer Best Practices & Golden Rules

1. **Never run compute jobs on Login Nodes**: Always use **sbatch** (for batch scripts) or **srun** (for interactive testing) to get assigned to a compute node.
2. **Be conservative with **--time** limits**: If your job takes 3 minutes, request **--time=00:10:00**. If you request **--time=24:00:00**, SLURM's scheduler will take much longer to find a slot for your job!
3. **Always inspect **.out** and **.err** files**: When a job disappears from **squeue**, run **cat job_<id>.out** and **cat job_<id>.err** to confirm results or diagnose Python tracebacks.
4. **Use Environment Modules**: Supercomputers use the **module** command to switch between compiler and CUDA versions:

```
module avail          # List available software packages
module load cuda/12.2 # Load CUDA Toolkit into PATH
module load openmpi    # Load OpenMPI libraries
```